

廈門大學



信息学院软件工程系

《JAVA 程序设计》实验报告

实验 2

姓名：王子恒

学号：34520242201240

学院：信息学院

专业：软件工程

完成时间：2026 年 3 月 9 日

一、实验目的及要求

实验目的

- 练习使用程序控制结构。

实验要求

- 按照题目要求编写代码。
- 撰写实验报告。
- 在下周实验课前，通过 course 系统提交源代码和实验报告。

二、实验题目及实现过程

1. 实验题目

- **基本题目：** 随机生成六个筛子点数，并根据博饼游戏规则判断得奖情况（需处理多奖项并存的情况）。
- **扩展题目：** 完善游戏逻辑，实现多玩家（6-10人）轮流摇骰，维护游戏奖项库存（Inventory），并在奖项发完后终止游戏并进行结算。

2. 实现过程与逻辑设计

在实现过程中，我主要采用了以下核心策略：

- **奖项判断逻辑 (`judgePrizes` 方法):**
 - 通过定义 `int[] counts` 数组记录 1-6 点数的出现次数，以便快速判断不同奖项的触发条件。
 - 针对六子博饼规则，利用 `IntStream` 对点数分布进行统计。例如，判断“对堂”时通过 `allMatch` 检查每个点数是否各出现一次；判断“状元”时分别处理了“四个四”和“五子”的情况。
 - 由于博饼允许“多奖并存”，代码中未使用 `if-else`，而是为每个奖项设置独立的 `if` 条件，确保单次投掷能同时触发多个符合规则的奖项。
- **游戏流程控制：**
 - 使用 `Map<Prize, Integer>` 作为全局 `INVENTORY`，严格按照题目要求的奖项份数进行库存管理。
 - 通过 `while(hasPrizes())` 循环控制游戏进程，确保在所有奖项被领完前持续进行，通过 `turn % playerCount` 实现玩家轮流投掷。
- **数据结构选择：**
 - 利用 Java 的 `record` 关键字定义 `Player` 类，简洁地存储每个玩家的 ID 及已获得的奖项记录。

- 投掷骰子部分使用了 `Random.ints(6, 1, 7)` 快速生成符合范围的随机数。

3.运行结果

请输入玩家人数(6-10): 8

---博饼开始---

玩家1摇出[6, 5, 5, 4, 2, 1] → 获得 [一秀] !
玩家2摇出[6, 5, 4, 4, 3, 3] → 获得 [二举] !
玩家3摇出[6, 6, 6, 5, 4, 2] → 获得 [一秀] !
玩家4摇出[6, 6, 3, 2, 1, 1] → 未中奖
玩家5摇出[6, 4, 3, 2, 2, 1] → 获得 [一秀] !
玩家6摇出[6, 6, 4, 2, 2, 1] → 获得 [一秀] !
玩家7摇出[6, 6, 6, 6, 3, 1] → 未中奖
玩家8摇出[6, 5, 5, 2, 1, 1] → 未中奖
玩家1摇出[6, 4, 4, 2, 2, 1] → 获得 [二举] !
玩家2摇出[3, 3, 3, 2, 2, 1] → 未中奖
玩家3摇出[6, 5, 4, 1, 1, 1] → 获得 [一秀] !
玩家4摇出[5, 3, 3, 3, 3, 1] → 获得 [四进] !
玩家5摇出[6, 5, 5, 2, 2, 1] → 未中奖
玩家6摇出[4, 3, 3, 2, 2, 1] → 获得 [一秀] !
玩家7摇出[6, 4, 3, 3, 2, 1] → 获得 [一秀] !
玩家8摇出[6, 4, 3, 2, 1, 1] → 获得 [一秀] !
玩家1摇出[6, 4, 4, 4, 3, 3] → 获得 [三红] !
玩家2摇出[5, 3, 3, 2, 2, 1] → 未中奖
玩家3摇出[5, 5, 4, 4, 3, 2] → 获得 [二举] !
玩家4摇出[6, 5, 5, 3, 3, 1] → 未中奖
玩家5摇出[6, 6, 5, 5, 4, 1] → 获得 [一秀] !
玩家6摇出[6, 6, 5, 3, 3, 2] → 未中奖
玩家7摇出[6, 4, 4, 3, 3, 2] → 获得 [二举] !
玩家8摇出[6, 5, 4, 3, 1, 1] → 获得 [一秀] !
玩家1摇出[6, 6, 4, 4, 3, 1] → 获得 [二举] !
玩家2摇出[5, 3, 1, 1, 1, 1] → 获得 [四进] !
玩家3摇出[5, 4, 4, 4, 3, 2] → 获得 [三红] !
玩家4摇出[6, 6, 5, 5, 2, 1] → 未中奖
玩家5摇出[6, 6, 4, 3, 3, 3] → 获得 [一秀] !
玩家6摇出[5, 5, 3, 2, 2, 2] → 未中奖
玩家7摇出[5, 4, 4, 4, 4, 1] → 获得 [四红] 状元!
玩家8摇出[6, 4, 3, 2, 1, 1] → 获得 [一秀] !
玩家1摇出[6, 4, 3, 3, 2, 1] → 获得 [一秀] !
玩家2摇出[5, 5, 4, 3, 3, 3] → 获得 [一秀] !
玩家3摇出[6, 6, 6, 4, 1, 1] → 获得 [一秀] !
玩家4摇出[4, 4, 3, 3, 3, 2] → 获得 [二举] !
玩家5摇出[5, 4, 4, 4, 2, 1] → 获得 [三红] !

玩家6摇出[6, 6, 5, 3, 2, 1] → 未中奖
玩家7摇出[4, 3, 3, 3, 2, 2] → 获得【一秀】！
玩家8摇出[6, 6, 4, 3, 3, 2] → 获得【一秀】！
玩家1摇出[6, 6, 5, 5, 4, 2] → 获得【一秀】！
玩家2摇出[5, 3, 3, 3, 2, 1] → 未中奖
玩家3摇出[6, 6, 5, 4, 3, 1] → 获得【一秀】！
玩家4摇出[5, 5, 4, 4, 4, 1] → 获得【三红】！
玩家5摇出[6, 5, 2, 2, 1, 1] → 未中奖
玩家6摇出[5, 3, 2, 2, 2, 1] → 未中奖
玩家7摇出[6, 6, 4, 3, 1, 1] → 获得【一秀】！
玩家8摇出[6, 4, 3, 2, 1, 1] → 获得【一秀】！
玩家1摇出[6, 6, 5, 4, 4, 2] → 获得【二举】！
玩家2摇出[6, 5, 5, 4, 2, 1] → 获得【一秀】！
玩家3摇出[4, 4, 3, 2, 1, 1] → 获得【二举】！
玩家4摇出[6, 5, 4, 3, 2, 1] → 获得【对堂】！
玩家4摇出[6, 5, 4, 3, 2, 1] → 获得【一秀】！
玩家5摇出[6, 5, 5, 4, 3, 2] → 获得【一秀】！
玩家6摇出[6, 5, 5, 4, 3, 2] → 获得【一秀】！
玩家7摇出[6, 6, 6, 4, 3, 1] → 获得【一秀】！
玩家8摇出[6, 5, 5, 5, 3, 2] → 未中奖
玩家1摇出[6, 4, 3, 2, 2, 2] → 获得【一秀】！
玩家2摇出[5, 5, 3, 2, 1, 1] → 未中奖
玩家3摇出[6, 4, 4, 2, 1, 1] → 获得【二举】！
玩家4摇出[6, 2, 2, 2, 1, 1] → 未中奖
玩家6摇出[4, 4, 3, 2, 2, 2] → 获得【二举】！
玩家7摇出[6, 4, 4, 3, 3, 3] → 获得【二举】！
玩家8摇出[5, 5, 4, 2, 1, 1] → 获得【一秀】！
玩家1摇出[6, 6, 5, 4, 4, 1] → 获得【二举】！
玩家2摇出[6, 5, 4, 4, 3, 1] → 获得【二举】！
玩家3摇出[6, 5, 4, 4, 3, 2] → 获得【二举】！
玩家4摇出[6, 5, 4, 3, 1, 1] → 获得【一秀】！
玩家5摇出[6, 6, 2, 2, 1, 1] → 未中奖
玩家6摇出[4, 3, 2, 2, 2, 1] → 获得【一秀】！
玩家7摇出[6, 5, 5, 4, 1, 1] → 获得【一秀】！
玩家8摇出[6, 5, 3, 3, 2, 1] → 未中奖
玩家1摇出[6, 4, 3, 2, 1, 1] → 获得【一秀】！
玩家2摇出[6, 6, 5, 4, 4, 1] → 获得【二举】！
玩家4摇出[6, 6, 6, 5, 3, 2] → 未中奖
玩家5摇出[6, 4, 4, 4, 4, 1] → 获得【四红】 状元！
玩家6摇出[6, 6, 4, 4, 3, 1] → 获得【二举】！
玩家3摇出[5, 5, 5, 3, 2, 2] → 未中奖
玩家4摇出[6, 6, 6, 5, 2, 2] → 未中奖
玩家5摇出[6, 6, 3, 3, 3, 2] → 未中奖
玩家6摇出[6, 6, 3, 2, 2, 2] → 未中奖

玩家2摇出[6, 6, 5, 3, 3, 1] → 未中奖
玩家4摇出[6, 5, 2, 2, 1, 1] → 未中奖
玩家3摇出[6, 6, 6, 3, 3, 1] → 未中奖
玩家4摇出[6, 5, 5, 3, 3, 1] → 未中奖
玩家5摇出[6, 6, 6, 6, 3, 2] → 未中奖
玩家6摇出[6, 5, 3, 3, 2, 2] → 未中奖
玩家1摇出[6, 6, 3, 3, 3, 1] → 未中奖
玩家2摇出[6, 5, 5, 5, 3, 2] → 未中奖
玩家3摇出[6, 5, 3, 2, 1, 1] → 未中奖
玩家1摇出[6, 6, 5, 5, 3, 3] → 未中奖
玩家5摇出[6, 5, 5, 2, 2, 2] → 未中奖
玩家8摇出[6, 5, 3, 3, 2, 1] → 未中奖
玩家1摇出[6, 6, 5, 3, 3, 1] → 未中奖
玩家2摇出[5, 5, 2, 1, 1, 1] → 未中奖
玩家4摇出[5, 3, 3, 2, 2, 2] → 未中奖
玩家5摇出[6, 5, 3, 3, 1, 1] → 未中奖
玩家8摇出[6, 5, 3, 2, 2, 1] → 未中奖
玩家2摇出[5, 3, 3, 2, 2, 1] → 未中奖
玩家3摇出[6, 6, 5, 5, 5, 2] → 未中奖
玩家1摇出[6, 6, 5, 2, 1, 1] → 未中奖
玩家3摇出[6, 5, 5, 3, 2, 1] → 未中奖
玩家4摇出[6, 6, 3, 3, 1, 1] → 未中奖
玩家7摇出[6, 6, 6, 5, 1, 1] → 未中奖
玩家6摇出[6, 6, 6, 3, 2, 2] → 未中奖
玩家3摇出[5, 5, 5, 3, 2, 1] → 未中奖
玩家6摇出[6, 5, 5, 3, 3, 3] → 未中奖
玩家3摇出[6, 5, 4, 3, 2, 1] → 获得【对堂】！
玩家4摇出[6, 5, 3, 2, 2, 2] → 未中奖
玩家6摇出[6, 6, 5, 3, 1, 1] → 未中奖
玩家1摇出[6, 3, 2, 2, 1, 1] → 未中奖
玩家2摇出[6, 6, 6, 5, 5, 2] → 未中奖
玩家5摇出[5, 3, 3, 1, 1, 1] → 未中奖
玩家6摇出[6, 6, 5, 2, 2, 1] → 未中奖
玩家1摇出[5, 5, 3, 2, 2, 1] → 未中奖
玩家6摇出[5, 5, 5, 3, 2, 1] → 未中奖
玩家4摇出[6, 6, 3, 3, 2, 1] → 未中奖
玩家7摇出[6, 5, 3, 3, 3, 1] → 未中奖
玩家1摇出[6, 5, 3, 3, 3, 1] → 未中奖
玩家2摇出[5, 5, 3, 3, 3, 1] → 未中奖
玩家3摇出[6, 3, 3, 3, 3, 2] → 获得【四进】！
玩家7摇出[6, 6, 5, 5, 1, 1] → 未中奖
玩家8摇出[6, 4, 3, 3, 3, 3] → 获得【四进】！
玩家1摇出[6, 5, 5, 3, 2, 1] → 未中奖
玩家8摇出[6, 6, 2, 2, 1, 1] → 未中奖

玩家2摇出[6, 6, 6, 5, 3, 2] → 未中奖
玩家4摇出[6, 5, 3, 2, 1, 1] → 未中奖
玩家6摇出[6, 5, 3, 2, 2, 1] → 未中奖
玩家1摇出[3, 3, 3, 2, 2, 1] → 未中奖
玩家3摇出[6, 5, 2, 2, 1, 1] → 未中奖
玩家5摇出[3, 3, 3, 2, 1, 1] → 未中奖
玩家1摇出[6, 6, 6, 3, 2, 2] → 未中奖
玩家2摇出[6, 3, 3, 2, 1, 1] → 未中奖
玩家4摇出[6, 6, 5, 3, 2, 2] → 未中奖
玩家6摇出[6, 6, 4, 4, 4, 4] → 获得【四红】状元!
玩家1摇出[4, 3, 3, 3, 3, 1] → 获得【四进】!
玩家8摇出[6, 6, 5, 5, 5, 3] → 未中奖
玩家3摇出[6, 5, 5, 3, 2, 1] → 未中奖
玩家5摇出[6, 5, 3, 2, 2, 1] → 未中奖
玩家1摇出[5, 3, 3, 3, 1, 1] → 未中奖
玩家5摇出[6, 3, 3, 1, 1, 1] → 未中奖
玩家7摇出[6, 5, 3, 3, 2, 1] → 未中奖
玩家8摇出[6, 6, 6, 5, 5, 2] → 未中奖
玩家2摇出[5, 5, 3, 2, 2, 1] → 未中奖
玩家4摇出[6, 3, 3, 3, 2, 1] → 未中奖
玩家6摇出[5, 3, 2, 1, 1, 1] → 未中奖
玩家8摇出[6, 5, 5, 2, 2, 1] → 未中奖
玩家2摇出[6, 6, 6, 5, 1, 1] → 未中奖
玩家3摇出[6, 3, 2, 2, 1, 1] → 未中奖
玩家6摇出[6, 5, 5, 5, 2, 1] → 未中奖
玩家8摇出[6, 6, 2, 2, 1, 1] → 未中奖
玩家3摇出[6, 6, 5, 5, 3, 1] → 未中奖
玩家4摇出[6, 4, 4, 4, 4, 3] → 获得【四红】状元!
玩家7摇出[6, 6, 3, 1, 1, 1] → 未中奖
玩家2摇出[5, 5, 5, 2, 2, 1] → 未中奖
玩家3摇出[6, 5, 2, 2, 1, 1] → 未中奖
玩家8摇出[5, 3, 1, 1, 1, 1] → 获得【四进】!
玩家1摇出[6, 6, 2, 2, 2, 1] → 未中奖
玩家4摇出[6, 6, 6, 3, 3, 1] → 未中奖
玩家5摇出[6, 3, 3, 2, 1, 1] → 未中奖
玩家7摇出[6, 6, 5, 5, 3, 1] → 未中奖
玩家2摇出[6, 6, 5, 5, 2, 1] → 未中奖
玩家5摇出[5, 3, 2, 2, 1, 1] → 未中奖
玩家6摇出[3, 3, 3, 1, 1, 1] → 未中奖
玩家7摇出[5, 5, 3, 3, 2, 2] → 未中奖
玩家1摇出[6, 5, 5, 3, 1, 1] → 未中奖
玩家3摇出[6, 6, 5, 5, 3, 1] → 未中奖
玩家4摇出[6, 2, 2, 1, 1, 1] → 未中奖
玩家8摇出[6, 3, 3, 3, 3, 2] → 获得【四进】!

玩家1摇出[6, 5, 2, 2, 1, 1] → 未中奖
玩家4摇出[5, 3, 2, 1, 1, 1] → 未中奖
玩家5摇出[5, 3, 2, 2, 2, 1] → 未中奖
玩家7摇出[6, 6, 2, 1, 1, 1] → 未中奖
玩家8摇出[6, 6, 5, 2, 2, 1] → 未中奖
玩家8摇出[6, 5, 5, 3, 1, 1] → 未中奖
玩家5摇出[5, 5, 3, 1, 1, 1] → 未中奖
玩家2摇出[5, 5, 3, 2, 1, 1] → 未中奖
玩家3摇出[4, 4, 4, 4, 4, 2] → 获得【五红】状元！
玩家5摇出[5, 3, 3, 2, 1, 1] → 未中奖
玩家7摇出[3, 3, 3, 2, 2, 2] → 未中奖
玩家1摇出[6, 6, 3, 1, 1, 1] → 未中奖
玩家2摇出[5, 5, 2, 2, 1, 1] → 未中奖
玩家5摇出[6, 5, 5, 5, 1, 1] → 未中奖
玩家6摇出[3, 3, 1, 1, 1, 1] → 获得【四进】！
玩家7摇出[5, 5, 3, 3, 1, 1] → 未中奖
玩家8摇出[6, 6, 6, 1, 1, 1] → 未中奖

--- 最后的状态为3获得，状元类型：五红 ---

--- 游戏结束，结算成绩 ---

玩家1获得的博饼结果：四进x1，一秀x5，二举x4，三红x1
玩家2获得的博饼结果：一秀x2，四进x1，二举x3
玩家3获得的博饼结果：四进x1，一秀x4，二举x4，三红x1，对堂x1，状元x1
玩家4获得的博饼结果：一秀x2，四进x1，二举x1，三红x1，对堂x1
玩家5获得的博饼结果：一秀x4，三红x1
玩家6获得的博饼结果：四进x1，一秀x4，二举x2
玩家7获得的博饼结果：一秀x5，二举x2
玩家8获得的博饼结果：四进x3，一秀x6

三、实验总结与心得记录

1. 实验核心逻辑总结

本实验通过 Java 编程完整实现了博饼游戏的数字化模拟。在实现过程中，我重点解决了以下技术问题：

- **多奖项判定的严谨性：**博饼规则中存在“多奖并存”的情况（如“四进带一秀”）。我通过使用多个独立的 `if` 分支而非 `if-else` 结构，确保了程序能够捕获单次投掷中产生的所有合法奖项。
- **游戏生命周期管理：**利用 `Map` 容器维护奖项库存，并结合 `while` 循环和取模运算（`turn % pLayerCount`）实现了多玩家公平轮流投掷，直到奖项库存耗尽为止，保证了游戏逻辑的完整性。

2. Java 技术应用心得

- **函数式编程的便利：**在统计骰子点数和判断奖项时，我大量使用了 `IntStream` 和 `Lambda` 表达式。相比传统的嵌套循环，`anyMatch`、`allMatch` 以及 `filter` 等方法显著提高了代码的可读性和开发效率。
- **新特性尝试：**实验中尝试使用了 Java 16+ 引入的 `record` 类型来定义玩家数据结构，这使得数据载体类的编写变得极其简洁，避免了冗长的 Getter 和 Setter 代码。
- **解耦设计：**将奖项判断逻辑抽离到 `judgePrizes` 静态方法中，实现了逻辑与显示的解耦，便于后续对规则进行维护或扩展。

3. 实验反思与改进

通过本次实验，我深刻体会到在处理具有复杂规则的游戏逻辑时，**数据结构的设计**至关重要。最初在处理奖项库存时曾考虑使用数组，但最终选择 `HashMap` 是因为它在处理“名称-剩余数量”这类映射关系时更加直观且易于维护。

此外，实验也提醒我在编写交互式程序时，必须考虑到**用户输入的健壮性**，例如在获取玩家人数时加入了类型检查和范围限制，以防止程序因非法输入而崩溃。